

ISLERI CP Communication Protocol

CP/1.0

Formal Draft Specification

Status: Draft — Architecture Finalized

Revision: 1.0-D1

Date: 2026

Organization: ISLERI

Primary implementation: CerePhyte 4.2

Controller: ATmega328P-class controller

Shield bus: RS485

Controller–ESP interface: I²C

Copyright © 2026 Omm Prakash Das / ISLERI.

This specification is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0). Implementations of the ISLERI CP protocol are not required to be licensed under CC BY-SA solely because they implement this specification.

1. Scope

The ISLERI CP Communication Protocol (CP/1.0) defines the communication architecture used between an ISLERI central controller and modular shields.

The protocol is designed so that a shield can introduce new functionality without requiring shield-specific modifications to the central controller firmware.

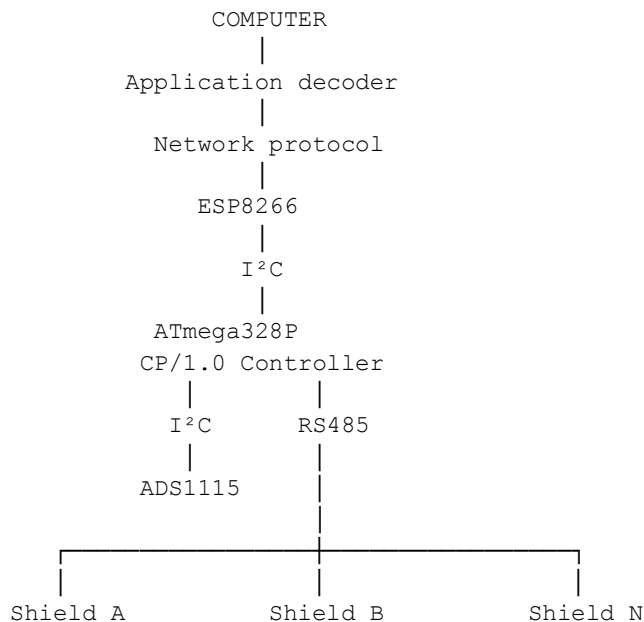
- scientific sensors;
- electrophysiology instrumentation;
- high-speed data acquisition;
- environmental sensors;
- actuators;
- motor controllers;
- displays;
- storage devices;
- signal-processing modules;
- experimental devices;
- future devices whose application data is unknown when the controller firmware is developed.

CP/1.0 is intentionally device-agnostic.

2. Fundamental Architecture

CP/1.0 follows four principles:

Fixed communication envelope. Flexible payload. Generic controller. Intelligent endpoints.



3. Normative Language

- MUST — mandatory for compliance.
- MUST NOT — prohibited.
- SHOULD — recommended unless there is a justified implementation reason.
- SHOULD NOT — normally prohibited unless justified.

- MAY — optional.
- RESERVED — must not be used without future protocol allocation.

4. Physical Transport

4.1 RS485

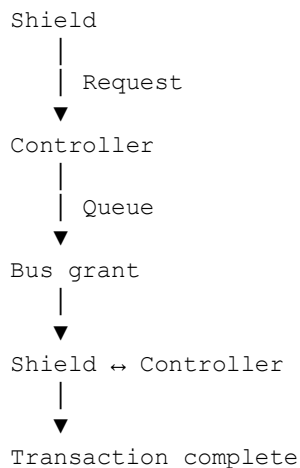
The CP shield network uses a half-duplex RS485 bus.

Parameter	Value
Physical layer	RS485
Mode	Half-duplex
Target baud rate	1,000,000 baud
UART format	8-N-1
Bus ownership	Controller-managed
Simultaneous transmitters	1 maximum

The 1 Mbps value is the CP/1.0 target and must be validated against the final transceiver, PCB, cable and hub implementation.

5. Bus Ownership

The RS485 network is a shared communication medium. Only one device may transmit at any moment.



Shields MUST NOT independently begin arbitrary RS485 transmissions.

6. Shield Parallelism

CP/1.0 does not require shields to stop their internal operation while another device communicates.

- acquire data;
- process data;
- operate actuators;
- cache results;
- prepare responses;

independently of RS485 bus availability. Communication is serialized. Device operation is not.

7. Device Addressing

Every shield **MUST** have a 16-bit Device ID.

Range	Purpose
0x0000	Controller
0x0001–0x7FFF	Standard device IDs
0x8000–0xFFEF	Extended/device-assigned IDs
0xFFFF0–0xFFFFE	Reserved
0xFFFF	Broadcast

The address identifies an endpoint and **MUST NOT** inherently encode the device's function.

8. Broadcast

0xFFFF represents broadcast. A controller **MAY** broadcast discovery, configuration or other permitted messages. A shield receiving a broadcast **MUST NOT** automatically transmit a response unless the message explicitly permits responses.

9. CP/1.0 Frame

SYNC 2 B	VER 1 B	FLAGS 1 B	SOURCE 2 B	DEST 2 B	TYPE 1 B	SEQ 2 B	LENGTH 2 B
TRANSACTION ID – 4 B							
PAYLOAD 0–220 bytes							
CRC-16 – 2 B							

Maximum frame size is 256 bytes.

10. Synchronization

CP/1.0 uses SYNC = 0xA5 0x5A. Synchronization alone does not make a frame valid. A frame must additionally pass version validation, length validation, structural validation and CRC validation.

11. Protocol Version

The version field is one byte. CP/1.0 is represented as 0x10. The upper nibble represents the major version and the lower nibble represents the minor version.

0x10 → CP/1.0

0x11 → CP/1.1

0x20 → CP/2.0

12. Flags

Bit	Meaning
0	ACK requested

1	ACK frame
2	Fragmented
3	Priority transaction
4	Response
5	Command
6	Reserved / future security use
7	Reserved

Reserved bits MUST be transmitted as zero unless a future specification defines them.

13. Source and Destination

SOURCE identifies the sender. DEST identifies the intended receiver.

```
SOURCE = 0x0000
DEST    = 0x0027
```

means: Controller → Device 0x0027

14. Message Types

ID	Type
0x01	METADATA
0x02	COMMAND
0x03	RESPONSE
0x04	DATA
0x05	TELEMETRY
0x06	EVENT
0x07	STREAM
0x08	CONFIG
0x09	ACK
0x0A	ERROR
0x0B	HEARTBEAT
0x0C	DISCOVERY

0x80–0xEF are available for future standardized message types. 0xF0–0xFF are reserved for experimental/vendor-specific extensions.

15. Sequence Numbers

Every streaming/data-producing endpoint SHOULD maintain a 16-bit sequence number. Sequence numbers wrap from 65535 to 0.

16. Transaction IDs

A 32-bit Transaction ID associates related operations. Transaction IDs are independent from sequence numbers.

Sequence number identifies position in a data stream. Transaction ID identifies a logical operation.

17. Integrity

CP/1.0 uses CRC-16/CCITT-FALSE.

```
Polynomial = 0x1021
Initial    = 0xFFFF
RefIn      = false
RefOut     = false
XorOut     = 0x0000
```

The CRC covers the defined frame contents excluding the synchronization bytes. A frame failing CRC validation **MUST NOT** be delivered as valid application data.

18. Payload Size

The maximum CP/1.0 frame is 256 bytes. The maximum payload is 220 bytes. A device requiring more data **MUST** use fragmentation.

19. Payload Philosophy

CP/1.0 deliberately does not prescribe one universal application data structure.

- Structured payload — for normal application data.
- Array/stream payload — for large collections of homogeneous values.
- Opaque payload — arbitrary binary data whose application meaning is not known to the controller.

20. Typed Data

Structured payloads **MAY** contain typed values including unsigned integers, signed integers, floating-point values, Boolean values, strings, byte arrays, arrays, structures, maps and opaque binary data.

21. TLV

CP/1.0 structured payloads **SHOULD** use a Type-Length-Value structure.

TYPE	LENGTH	VALUE
------	--------	-------

This permits fields to be added without redesigning the complete structure.

22. Schema IDs

Application-specific payloads **SHOULD** identify their schema.

```
Schema ID      = 32-bit
Schema Version = 16-bit
```

Example:

```
Schema ID      = 0x83A21704
Schema Version = 0x0002
```

23. Unknown Payload Rule

The controller **MUST NOT** reject an otherwise valid frame merely because it does not understand the application's payload schema. If framing, protocol version, length and CRC are valid, the controller **SHOULD** transport the payload.

24. Fragmentation

Payloads larger than 220 bytes MUST be fragmented.

Message ID	4 B
Fragment Index	2 B
Total Fragments	2 B
Fragment Payload	

The ATmega SHOULD forward fragments without reconstructing the complete application object.

25. Data Streams

A high-rate device MAY transmit data using STREAM. A stream should contain information sufficient to determine source, sequence, sample/data format, count, timebase where required, and schema.

26. Timestamping

Time-sensitive data MAY contain timestamps. For sample blocks, a single timestamp plus a defined sampling rate/timebase SHOULD be preferred over placing a complete timestamp on every sample.

27. Discovery

After reset, the controller performs shield discovery.

```
RESET
↓
Initialize hardware
↓
Initialize RS485
↓
DISCOVERY
↓
Shield metadata
↓
Validate compatibility
↓
Build device table
↓
Configure priority ownership
↓
Configure hub mapping
↓
Normal operation
```

28. Shield Metadata

A CP/1.0-compliant shield MUST expose metadata sufficient for generic operation.

- Device ID;
- protocol version;
- hardware revision;
- firmware revision;
- capabilities;
- supported message types;

- maximum payload;
- schema identifiers;
- command capabilities;
- priority capability;
- hub requirements.

29. Capability Declaration

A shield **SHOULD** advertise capabilities rather than relying on the controller to identify it by a fixed device category.

```
CAN_SEND_DATA
CAN_RECEIVE_COMMANDS
CAN_STREAM
CAN_USE_PRIORITY
CAN_FRAGMENT
CAN_CACHE
CAN_ACTUATE
```

30. Normal Request Queue

Shields without dedicated priority inputs use the shared request mechanism. The controller maintains a request queue. The queue **SHOULD** use fair scheduling. The recommended CP/1.0 strategy is round-robin servicing.

31. Priority Attention

Two dedicated priority channels are available using PCINT-capable controller inputs.

Current CerePhyte mapping	Function
J2 Pin 4	Priority Input A
J2 Pin 6	Priority Input B

The logical owner of each line is determined during metadata exchange.

32. Equal Priority

Priority A and Priority B have equal priority. There is no permanent $A > B$ relationship. If A is already being serviced and B asserts, B cannot bypass A.

33. Persistent Priority Request

A priority-capable shield **MUST** keep its priority line asserted until the controller has serviced the associated priority request.

```
Shield detects event
↓
Assert PCINT
↓
Controller detects PCINT
↓
Identify shield
↓
```



```
Select hub channel if required
    ↓
Service transaction
    ↓
Transaction complete
    ↓
Shield releases PCINT
```

34. Priority Timeout

A priority transaction is limited to a default 50 ms service window. A shield requiring longer processing **SHOULD** provide the urgent information, release the priority line, cache remaining work and request another transaction later.

35. Simultaneous Priority Requests

If both priority lines become active before either transaction begins, the controller **SHALL** use a fair arbitration mechanism. CP/1.0 recommends first-detected request, followed by fair alternation when simultaneous detection cannot be distinguished. If one transaction is already active, it **MUST** complete before the other priority request is serviced.

36. Intelligent Hub

An ISLERI intelligent hub may allow multiple shields to connect to a single CerePhyte system. The controller maintains a Device ID → Hub Channel mapping.

```
0x0021 → Channel 3
0x0048 → Channel 7
0x0051 → Channel 2
```

37. J2 Pin 3

J2 Pin 3 is the logical Channel Select interface. The controller uses it to select a shield through a compatible intelligent hub. The exact electrical encoding of the channel is a hub implementation parameter and is not frozen by the CP transport protocol.

38. J2 Pins 4 and 6

J2 Pin 4 and J2 Pin 6 are reserved as the two priority inputs. The actual shield assigned to each line is established through metadata exchange.

39. Commands

CP/1.0 is fully bidirectional. A controller can send **COMMAND** to a shield. The shield may return **RESPONSE**, **TELEMETRY** or **EVENT** messages. The controller does not need to understand the application meaning of the command.

40. Command Safety

Commands affecting physical outputs **SHOULD** contain sufficient application-level information for the receiving shield to validate them. The receiving shield remains responsible for enforcing its own hardware safety limits.

41. ACK vs RESPONSE

ACK means the communication frame was successfully received/accepted at the protocol layer. RESPONSE means the device has produced an application-level result. Not every DATA or TELEMETRY frame requires an ACK.

42. Error Handling

CP/1.0 distinguishes transport errors from application errors.

- Transport: invalid synchronization; invalid protocol version; invalid length; CRC failure; timeout; unknown destination; bus failure.
- Application: unsupported command; invalid command parameter; device fault; unavailable resource; actuator failure.

A valid CP frame containing an application-level error is still a valid CP frame.

43. Flow Control

High-rate shields MUST NOT overwhelm the controller. A shield should be able to cache, pause, reduce transmission rate, wait for bus service and split data into fragments.

44. Controller Responsibilities

- ADC acquisition;
- RS485 management;
- device discovery;
- metadata handling;
- device addressing;
- priority handling;
- hub selection;
- frame validation;
- buffering;
- routing;
- generic data aggregation;
- controller-to-ESP transport.

It SHOULD NOT contain shield-specific application logic.

45. What the Controller Should Not Do

The controller should not contain hard-coded logic for individual shield types. Instead, it should receive, validate, identify, route, store and forward valid frames.

46. Generic Data Store

The controller's aggregate data structure should contain generic CP frames rather than application-specific variables.

```
FINAL_DATA
|
|— ADC frame
```

```

├─ Device 0x0010 frame
├─ Device 0x0021 frame
├─ Device 0x0034 frame
├─ Device 0x0047 response
├─ Device 0x0091 stream fragment
└─ ...

```

The computer separates and interprets the data later.

47. ADC Integration

CerePhyte ADC data originates through the controller's I²C-connected ADC devices. ADC data should be represented as CP-compatible generic data. A typical ADC stream may contain source, ADC identifier, channel, sequence, sample format, sample rate, sample count, timestamp/timebase and sample payload.

48. ATmega–ESP Boundary

The ESP8266 interface is intentionally outside the CP/1.0 shield-bus specification.

```

ADC + shields
  ↓
ATmega
  ↓
generic formatted data
  ↓
I2C
  ↓
ESP8266
  ↓
network protocol
  ↓
computer

```

The ESP does not need a separate decoder for every shield.

49. Power Optimization

CP/1.0 is designed with low-power operation in mind. The system should minimize unnecessary polling, unnecessary acknowledgements, repeated metadata transfers, idle RS485 activity and redundant data transmission. Shields SHOULD process and cache information locally.

50. ATmega Clock Modes

CerePhyte may operate the ATmega328P at 8 MHz normal mode and 16 MHz performance mode. The protocol is independent of clock frequency. The controller SHOULD switch to 16 MHz only when measured workload requires it. Potential indicators include RS485 processing utilization, buffer occupancy, queue latency, acquisition deadlines and missed service deadlines.

51. Example — Ordinary Sensor

```

Sensor
  ↓
Measurement
  ↓

```

```
Local processing
↓
Cache
↓
Request bus
↓
Controller grants access
↓
DATA
↓
Controller
↓
ESP
↓
Computer
```

52. Example — Motor Controller

```
Computer
↓
ESP
↓
ATmega
↓
COMMAND
↓
RS485
↓
Motor Shield
```

The motor shield performs the operation and may return RESPONSE, TELEMETRY and EVENT messages.

53. Example — High-Speed Acquisition Shield

A shield producing 5,000 bytes of data cannot fit this into one frame. It therefore uses 220-byte payload fragments. The computer reassembles the data.

54. Example — Unknown Future Shield

A future shield may send an unknown Schema ID. The CerePhyte controller should treat this as valid data if the CP frame itself is valid. Unknown schema is not automatically a protocol error.

55. Failure Scenario — Shield Disappears

If a known shield stops responding, the controller marks the device unavailable and continues operating other devices.

56. Failure Scenario — Bad CRC

A frame failing CRC is discarded and must not enter the application data stream as valid information.

57. Failure Scenario — Stuck Priority Line

If a priority input remains asserted beyond the allowed service window, the controller enters a priority fault condition and normal bus operation continues. The condition should be reported.

58. Failure Scenario — 50 Shields

Fifty devices do not transmit concurrently. They perform local processing and caching, submit requests, and are serviced through the fair queue. If measured CPU workload becomes excessive, the controller may enter 16 MHz performance mode.

59. Compatibility Requirements for Shields

- implement the CP/1.0 frame format;
- possess a valid Device ID;
- provide metadata;
- obey RS485 bus ownership;
- not transmit arbitrary unsolicited RS485 traffic;
- correctly implement CRC;
- obey maximum frame size;
- support the required message types it advertises;
- keep priority requests asserted until service if using a priority channel;
- cache data when necessary to tolerate bus delays;
- reject unsafe/invalid application commands itself.

60. Controller Compatibility Requirements

- discover shields;
- validate metadata;
- maintain Device IDs;
- manage RS485 bus ownership;
- validate frames;
- reject invalid CRC;
- support normal request scheduling;
- support the two priority channels;
- maintain priority ownership;
- support hub channel selection where a compatible hub exists;
- transport unknown valid payloads;
- support fragmentation;
- associate transactions;
- expose generic data to the upper layer.

61. Forward Compatibility

The protocol separates CP transport from application schema. New application schemas should not require a new CP protocol version.

62. Security Considerations

CP/1.0 is primarily an embedded local communication protocol. Network security between CerePhyte and external systems is outside this specification. CRC provides integrity checking but is not cryptographic security. Future secure command/authentication mechanisms may be added.

63. Reserved Space

- authentication;
- encryption;
- compressed payloads;
- advanced flow control;
- multicast;
- time synchronization;
- device firmware update;
- advanced diagnostics.

64. CP/1.0 Design Contract

Any ISLERI shield that conforms to CP/1.0 shall be able to communicate with a CP/1.0 controller through the defined communication architecture without requiring the controller to contain application-specific knowledge of that shield.

This does not mean every possible shield can be physically supported by every CerePhyte hardware revision. Electrical limitations, available power, bus loading, bandwidth and hub capacity remain hardware constraints. The communication model remains generic.

65. Final CP/1.0 Parameters

Parameter	CP/1.0
Physical bus	RS485
Bus mode	Half-duplex
Target speed	1 Mbps
UART	8-N-1
Device address	16-bit
Maximum frame	256 bytes
Maximum payload	220 bytes
Synchronization	A5 5A
CRC	CRC-16/CCITT-FALSE
Sequence	16-bit
Transaction ID	32-bit
Broadcast	0xFFFF
Normal scheduling	Fair round-robin
Priority inputs	2
Priority relation	Equal
Priority preemption	No
Priority request	Persistent
Default priority window	50 ms
Hub channel select	J2 Pin 3
Priority input A	J2 Pin 4
Priority input B	J2 Pin 6
Payload model	Extensible
Structured payload	TLV
Schema ID	32-bit

Fragmentation	Supported
Unknown payload	Transport unchanged
Controller interpretation	Generic
Application interpretation	Computer-side
Shield caching	Required design principle
Normal ATmega clock	8 MHz
Performance clock	16 MHz

66. Status of This Specification

CP/1.0-D1 is the finalized architectural draft for CerePhyte 4.2.

The following are implementation-validation items rather than unresolved architectural questions:

- exact RS485 transceiver selection;
- termination and bias resistor values;
- maximum validated cable length;
- exact physical maximum shield count;
- final intelligent-hub electrical design;
- exact J2 Pin 3 channel encoding;
- exact PCINT electrical polarity;
- final PCB signal-integrity validation;
- measured 8 MHz/16 MHz workload thresholds.

These values should be determined from the actual hardware rather than guessed in a protocol document.

67. One-Sentence Definition

ISLERI CP/1.0 is a controller-managed, RS485-based, bidirectional, extensible device communication protocol using fixed bounded frames, generic routing, transaction tracking, priority attention, fragmentation, schema-described payloads, and opaque-data forwarding so that new ISLERI shields can communicate without requiring shield-specific controller firmware.